All we need to do is specify the address of the device along with the quantity of bytes that the device should send. From our example code, it is as follows:

```
Wire.requestFrom(DS1307, 7);
```

Here we are addressing the `DS1307` device and requesting a total of 7 bytes to be sent. Once this data has been sent, we can use the `read()` function to do something with it.

read()

Now that we should have data waiting for us, we can use the `read()` function to pull in one byte of data at a time. Since there are no parameters for this function, and it only returns the next available byte to be read, let's look at the following example from our code:

```
second = bcdToDec(Wire.read());
```

In this statement, beginning at the far right, we are calling the `Wire.read()` function to read the first available byte from the slave device. Because this device sends data in BCD format, we need to convert this number to decimal and then assign the value to the variable `second`. We will later use these variables that contain the various units of time and print their values to the Serial Monitor.

And with that, we have wrapped up our whirlwind tour of I2C and the Wire library. It would be really nice if we had to the space to look even a small percentage of the I2C devices available to us, but sadly, we have to call it a day at some point.

Summary

In this chapter we had a good look at a few different forms of serial communications, from the hardware serial port on the Arduino microcontroller; to the software emulation of serial on any of the general purpose I/O pins; and I2C, a serial communication protocol for talking to various sensors, displays, and other devices. This was only the tip of the iceberg because there are even more communication protocols, like Serial Peripheral Interface, or SPI, and OneWire among others. Hopefully, though, this chapter has shown some of the similarities and a few of the differences between these few protocols to make learning new ones easier than before.

Speaking of learning new things, in the next chapter we are going to continue on with a loose collection of ideas for new projects, new hardware, and new programming languages. We'll even show a few ways that you might contribute back to the open hardware community by sharing what you've learned. With a solid framework for programming the Arduino firmly established, we will open up our discussion to include many of the things that we haven't yet been able to cover in depth. We'll show you some neat things and how they might be put to use, hopefully to inspire even greater projects than previously imagined. No more lengthy sketches and source code summaries, instead we will provide a brief overview of some of the many directions you might want to take with your newfound Arduino knowledge, and leave the rest in your capable hands.